

React AI E-Commerce Engine

Technical System Requirements, Backend Architecture, and Initialization Manual

Frontend:

React 18 / Vite / Tailwind

Backend:

Django REST Framework

Database:

MySQL 8.0+

Document Version:

2.0 (Full-Stack)

1. Executive Overview & Architectural Shift

The **React AI E-Commerce Platform** is an end-to-end intelligent shopping application. While the initial frontend specification focused on client-side vision search and state management, this updated full-stack specification integrates a high-performance Python **Django REST Framework (DRF)** backend coupled with a relational **MySQL** database.

To ensure platform security and session management, the primary system workflow mandates a **First-Page Authentication Gate**. Unauthenticated users are strictly routed to an integrated Login/Register view prior to accessing product catalogs, recommendations, or AI visual processing modules.

2. Full-Stack Technology Stack

Layer	Technology	Role & Integration Purpose
Frontend Core	React 18 + Vite	Fast client-side rendering, module bundling, and component-based UI architecture.
Frontend State	Redux Toolkit / Context	Global state management for user authentication session, cart persistence, and AI predictions.
Styling	Tailwind CSS	Utility-first responsive design framework for dark/light themes and dynamic components.
Backend Framework	Python Django / DRF	Robust API endpoint generation, ORM database abstraction, and custom middleware processing.
Database Engine	MySQL 8.0+	Relational storage for user profiles, product catalogs, order records, and taxonomy tags.
Security / Auth	JWT (SimpleJWT)	Stateless token-based authentication with Access and Refresh tokens.
AI / Computer Vision	TensorFlow.js / Canvas API	Client-side image feature extraction, visual product search, and instant tag matching.

3. Unified Full-Stack Project Structure

```
react-ai-ecommerce/  
├── frontend/  
│   ├── src/  
│   │   ├── assets/  
│   │   ├── components/  
│   │   ├── context/  
│   │   ├── features/  
│   │   ├── hooks/  
│   │   ├── pages/  
│   │   └── services/  
│   └── # React 18 + Vite Application  
│       ├── # Static assets, fallback imagery, branding  
│       ├── # UI Primitives (Navbar, Modals, Buttons, Cards)  
│       ├── # AuthContext for login state & JWT storage  
│       ├── # Business logic (auth, cart, vision-search)  
│       ├── # Custom hooks (useVision, usePredict, useAuth)  
│       ├── # Route views (Login, Register, Shop, Cart)  
│       └── # Axios interceptors & backend API client
```

```
├── ── utils/                                # Canvas formatters & vector calculation helpers
├── package.json
├── vite.config.js
├── backend/                                # Django REST Framework Backend
│   ├── manage.py
│   ├── core/                              # Master project settings
│   │   ├── settings.py                    # MySQL DB config, CORS headers, JWT settings
│   │   ├── urls.py                        # Global URL routing
│   │   └── wsgi.py
│   ├── apps/
│   │   ├── authentication/                # Custom User, JWT login/register endpoints
│   │   ├── products/                     # Catalogs, Categories, Inventory & Tags
│   │   └── ai_engine/                     # Vision model features & embedding storage
│   └── requirements.txt                    # Py dependencies (Django, djangorestframework, mysqlclient)
```

4. Authentication Entry Flow & Security Architecture

To ensure full protection of shopping and AI capabilities, the application enforces an explicit authorization boundary at root application launch:

- 1

Initial Route Check (`/`)
When a user visits the root web address, the React Router context queries local state and secure storage for an existing, valid JWT access token.
- 2

Unauthenticated Redirect Gate
If no token exists or if validation fails, the `ProtectedLayout` component blocks access to catalog pages and immediately redirects the user to `/login`.
- 3

Authentication Request (`POST /api/auth/login`)
The user submits credentials via the Login/Register form. Axios sends an encrypted payload to the Django backend.
- 4

MySQL Verification & Token Issuance
Django checks hashed password credentials against the MySQL `authentication\_user` table. Upon validation, SimpleJWT generates pair tokens (`access` and `refresh`).
- 5

Session Unlock & Dashboard Access
React stores the access token in memory/headers and unlocks route navigation to the main AI-powered e-commerce store (`/shop`, `/vision-search`).

5. Antigravity / Agent Initialization Prompt

Use the following structured prompt to bootstrap the full-stack codebase automatically within your AI development workspace:

```
Act as a Principal Full-Stack Engineer. Initialize a full-stack project based on the following specification:

1. ARCHITECTURE & TECH STACK:
  - Frontend: React 18 (Vite), Tailwind CSS, Redux Toolkit, Axios.
  - Backend: Python Django (Django REST Framework), MySQL 8.0+.
  - Authentication: JSON Web Tokens (SimpleJWT).

2. ENTRY POINT & AUTHENTICATION FLOW:
```

- Implement an immediate Authentication Gate. Unauthenticated users visiting '/' must be redirected to '/login'.
- Build dual Login and Register forms under `frontend/src/pages/`.
- Implement `AuthContext` and an Axios interceptor to automatically attach `Authorization: Bearer ` to requests.

3. MYSQL BACKEND CONFIGURATION:

- Configure Django `settings.py` for MySQL database connection (`ai\_ecommerce\_db`).
- Create custom User model, Product catalog model, Category model, and Tag model in `backend/apps/`.

4. AI-CENTRIC FRONTEND COMPONENTS:

- Implement Canvas-based image upload for visual search under `frontend/src/features/ai\_engine/`.
- Setup custom hook `useVision` utilizing TensorFlow.js for client-side embedding calculation.

5. DIRECTORY STRUCTURE:

- Follow the exact nested directory architecture outlined in Section 3 of this document.

Output the initialization configuration files (settings.py, React Router setup, and AuthContext).